

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

COURSE CODE: MLA01

CO2: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 Time: 90 Mins No. of Questions: 5

Annexure A

Algorithm Design Challenge

Code of Conduct:

I _____[S.Roshni\(192525185\)](#)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: [S.Roshni](#)

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	

AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency Originality, Critical Thinking, and Presentation	10% 10% 100%	
Total		

Signature of the Faculty Total Marks Awarded

Answer all the Questions

Q.No	Question	Topics
1	Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given RomaniaMap Problem graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.	Informed Search, Greedy Best-First Search, Heuristic Functions

2	A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.	A* Search, Heuristic Functions
3	A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.	CSP, MRV, Forward Checking
4	Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha–Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.	Game Playing, Minimax, Alpha–Beta Pruning

5	A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.	Local Search, Optimization, Online Search Agents
----------	---	---

Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	Problem Statement	Briefly describe the given problem in your own words.
2	Problem Analysis	Identify the initial state, goal state, assumptions, and constraints.
3	AI Technique Selection	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).

4	Justification of Algorithm	Explain why the selected algorithm is suitable compared to other search techniques.
5	State Space Representation	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	Variables, Domains, and Constraints (Applicable for CSP problems)	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	Heuristic Function (Applicable for Informed Search)	Define the heuristic function $h(n)$ and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	Algorithm Design / Pseudocode	Write the step-by-step algorithm or pseudocode for solving the problem.
9	Flowchart / Algorithm Diagram	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.

10	Working / Execution Trace	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	Complexity Analysis	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	Advantages and Limitations	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	Conclusion	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

Artificial Intelligence

Search Algorithms & CSP — Worked Examples

Student Name: S. Roshni(192525185)

Q1. Greedy Best-First Search — Romania Map

1. Problem Statement

Find a path from Arad to Bucharest using Greedy Best-First Search. The algorithm selects the node that appears closest to the goal using a heuristic.

2. Problem Analysis

- Initial State: Arad
- Goal State: Bucharest
- Heuristic: Straight-line distance to Bucharest
- Constraint: Move only through connected cities

3. AI Technique Selection

Greedy Best-First Search is used because it uses heuristic information to quickly move toward the goal.

$f(n) = h(n)$, where $h(n)$ is the estimated distance from node n to Bucharest.

4. Heuristic Values

Arad = 366, Sibiu = 253, Fagaras = 178, Bucharest = 0 (straight-line distance to Bucharest).

5. Algorithm / Pseudocode

- Put Arad in OPEN.
- Select the node having minimum $h(n)$.
- If the node is Bucharest, stop.
- Otherwise expand the node.
- Add unvisited successors to OPEN.
- Repeat until the goal is reached.

6. Working

Starting from Arad, the expansion order is: Arad → Sibiu → Fagaras → Bucharest.

Because $h(\text{Sibiu}) = 253 < h(\text{Timisoara}) = 329$, Sibiu is chosen. Then $h(\text{Fagaras}) = 178$ is chosen over Rimnicu Vilcea. Finally $h(\text{Bucharest}) = 0$, so Bucharest is selected.

7. Complexity

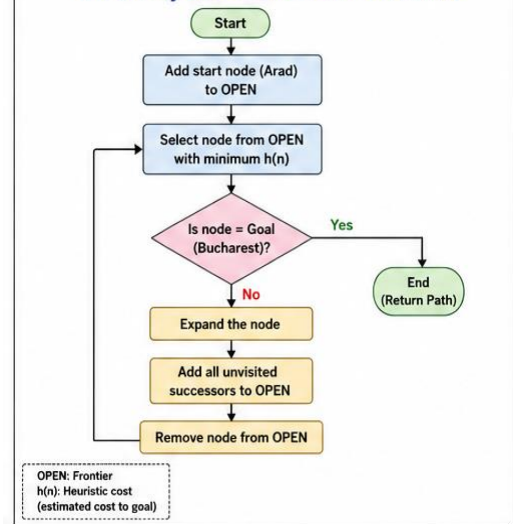
Time: $O(b^m)$ Space: $O(b^m)$ — where b is the branching factor and m is the maximum depth.

8. Advantages and Limitations

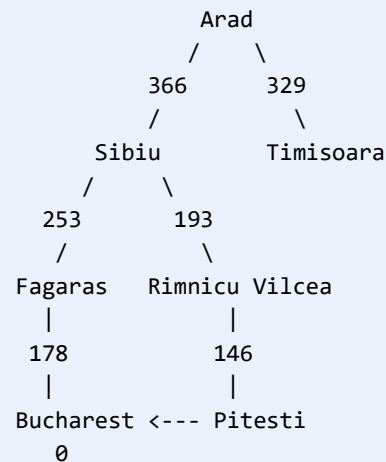
- Advantage: Fast and goal-directed.

Flowchart: Greedy Best-First Search

Q1. Greedy Best-First Search – Flowchart



State-Space: Romania Map



Heuristic Table $h(n)$

City	$h(n)$
Arad	366
Sibiu	253
Fagaras	178
Bucharest	0

- Advantage: Uses heuristic knowledge; simple to implement.
- Limitation: Does not guarantee the shortest path.
- Limitation: Depends on heuristic quality.

9. Conclusion

Greedy Best-First Search efficiently finds a path toward Bucharest by selecting the city with the smallest heuristic value. It is suitable when speed matters more than guaranteed optimality.

Q2. A* Search — Delivery Robot

1. Problem Statement

A delivery robot must travel from a Warehouse (W) to a Customer (G) through roads having different costs. A* is used to find the optimal path.

2. Problem Analysis

- Initial State: W
- Goal State: G
- Road cost: Different for each road
- Objective: Minimum-cost path

3. AI Technique Selection

A* Search is selected because it considers both the cost already travelled and the estimated remaining cost.

$f(n) = g(n) + h(n)$ where $g(n)$ = actual cost from start, $h(n)$ = estimated cost to goal.

4. Heuristic Values

W = 8, A = 6, B = 3, D = 2, G = 0.

5. Algorithm / Pseudocode

- Put Start in OPEN.
- Calculate $f(n) = g(n) + h(n)$.
- Select node with minimum $f(n)$.
- If node = Goal, return path.
- Expand the node.
- Calculate g , h and f for successors.
- Update OPEN with better paths.
- Repeat until Goal is reached.

6. Working

W: $g=0$, $h=8$, $f=8$. A: $g=2$, $h=6$, $f=8$. D: $g=6$, $h=2$, $f=8$. G: $g=8$, $h=0$, $f=8$.

Therefore the path is $W \rightarrow A \rightarrow D \rightarrow G$, with total cost 8.

7. Why A* is Optimal

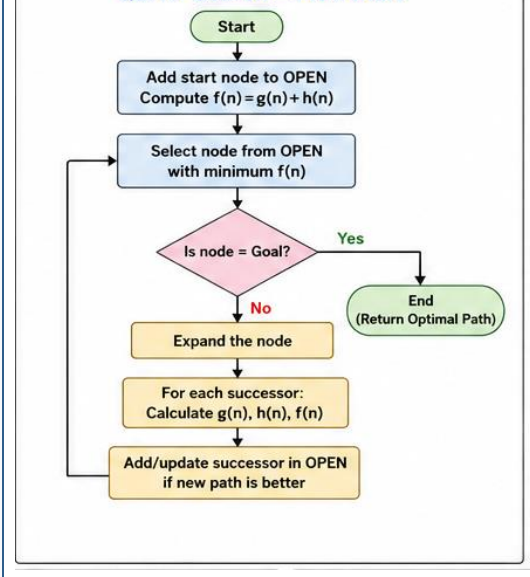
Path Chosen

Arad $\rightarrow$ Sibiu $\rightarrow$ Fagaras
 $\rightarrow$ Bucharest

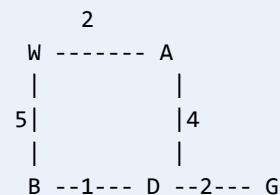
(goal reached, $h = 0$)

Flowchart: A* Search

Q2. A* Search – Flowchart



State-Space: Road Network



Heuristic Table $h(n)$

Node	$h(n)$
W	8
A	6
B	3
D	2
G	0

A* guarantees an optimal solution when the heuristic is admissible, meaning $h(n) \leq h^*(n)$ — it never overestimates the actual remaining cost.

8. Complexity

Time: $O(b^d)$ Space: $O(b^d)$

9. Advantages and Limitations

- Advantage: Finds the optimal path.
- Advantage: Uses both actual and estimated cost; more reliable than Greedy Search.
- Limitation: Requires high memory.
- Limitation: Performance depends on heuristic quality.

10. Conclusion

A* finds the minimum-cost delivery route $W \rightarrow A \rightarrow D \rightarrow G$ with total cost 8.

Q3. Examination Timetable Using CSP

1. Problem Statement

A university needs to prepare an examination timetable where subjects having common students must not be scheduled at the same time. This is modeled as a Constraint Satisfaction Problem: $CSP = (X, D, C)$.

2. Problem Analysis

- Variables: $X = \{AI, DBMS, DS, CD, MATH\}$
- Domains: $D = \{T1, T2, T3, T4\}$ (time slots)
- Constraints: conflicting subjects cannot share a slot, e.g. $AI \neq DBMS, AI \neq DS, DBMS \neq CD, DS \neq CD$

3. AI Technique Selection

Use CSP with MRV (Minimum Remaining Values), Forward Checking, and Backtracking.

4. MRV — Minimum Remaining Values

MRV selects the variable with the smallest available domain. Given $AI = \{T1, T2, T3, T4\}$, $DBMS = \{T1, T2\}$, $DS = \{T1, T3\}$, $CD = \{T2\}$, MRV selects CD because it has only one possible value.

5. Forward Checking

When a subject is assigned a slot, that slot is removed from the domains of all conflicting subjects. Example: $AI = T1$, so T1 is removed from DBMS, DS and CD. This prevents invalid assignments early.

6. Algorithm / Pseudocode

- If all subjects are assigned, return SUCCESS.
- Select an unassigned subject using MRV.
- Select a valid time slot.

$$f(n) = g(n) + h(n)$$

$$W: 0+8=8$$

$$A: 2+6=8$$

$$D: 6+2=8$$

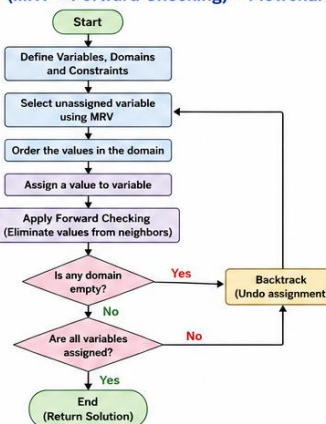
$$G: 8+0=8$$

Path: $W \rightarrow A \rightarrow D \rightarrow G$

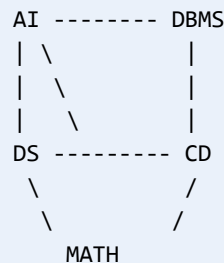
Total cost = 8

Flowchart: CSP (MRV + Forward Checking)

Q3. CSP (MRV + Forward Checking) – Flowchart



Constraint Graph



(edge = conflict)

Final Timetable

- Check constraints.
- Assign the slot.
- Apply Forward Checking.
- If no domain becomes empty, continue.
- If failure occurs, backtrack.
- Repeat until all subjects are assigned.

7. Complexity

For n subjects and d slots — Time: $O(d^n)$ Space: $O(n)$. MRV and Forward Checking reduce the practical search space.

8. Advantages and Conclusion

- Advantage: Handles complex constraints.
- Advantage: MRV reduces unnecessary search.
- Advantage: Forward Checking detects conflicts early.

Conclusion: CSP with MRV and Forward Checking provides an efficient method for generating a valid university examination timetable.

Q4. Minimax and Alpha-Beta Pruning

1. Problem Statement

A two-player game contains MAX and MIN players. MAX wants the highest score, MIN wants the lowest score. Minimax determines the best move for MAX.

2. AI Technique Selection

Minimax is used for optimal decision-making in two-player games. Alpha-Beta Pruning improves Minimax by removing branches that cannot affect the final result.

3. Minimax Algorithm

- If node is terminal, return its value.
- If node is MAX, return the maximum value of its children.
- If node is MIN, return the minimum value of its children.
- Continue recursively.

4. Working

First MAX node: $\text{MAX}(3,5) = 5$. Second MAX node: $\text{MAX}(6,9) = 9$. Left MIN: $\text{MIN}(5,9) = 5$.

Right side: $\text{MAX}(1,2) = 2$, $\text{MAX}(0,-1) = 0$. Right MIN: $\text{MIN}(2,0) = 0$. Root: $\text{MAX}(5,0) = 5$.

Therefore Best Move = Left, Minimax Value = 5.

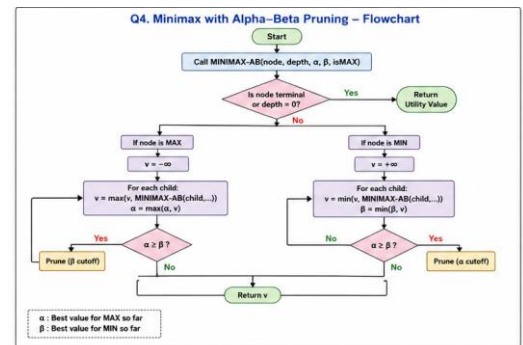
5. Alpha-Beta Pruning

Alpha-Beta uses α = best value for MAX and β = best value for MIN. Pruning occurs when $\alpha \geq \beta$.

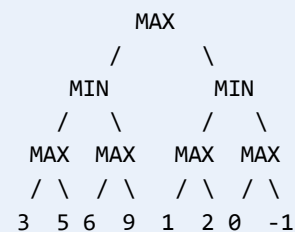
Subject	Slot
CD	T1
AI	T2
DBMS	T3
DS	T1
MATH	T2

No conflicting subjects share a slot.

Flowchart: Minimax with Alpha-Beta Pruning



Game Tree



Minimax Rollup

$\text{MAX}(3,5)=5$ $\text{MAX}(6,9)=9$
 $\text{MIN}(5,9)=5$

$\text{MAX}(1,2)=2$ $\text{MAX}(0,-1)=0$
 $\text{MIN}(2,0)=0$

After obtaining 5 from the left branch, the leaf 9 in the right part of that MIN subtree becomes unnecessary. Similarly, after obtaining 2 in the right subtree with root $\alpha = 5$, the leaf -1 is unnecessary.

6. Comparison

- Minimax nodes evaluated: 15
- Alpha-Beta nodes evaluated: 13
- Nodes pruned: 2 — both methods return the same value, 5

7. Complexity

Minimax: $O(b^d)$. Alpha-Beta worst case: $O(b^d)$. Alpha-Beta best case: $O(b^{(d/2)})$.

8. Advantages and Conclusion

Minimax gives optimal decisions but may evaluate many nodes. Alpha-Beta gives the same optimal decision while evaluating fewer nodes. Alpha-Beta Pruning improves Minimax efficiency.

Root: $\text{MAX}(5, 0) = 5$

Alpha-Beta Pruned Leaves

9 -> PRUNED
-1 -> PRUNED

Minimax nodes: 15
Alpha-Beta nodes: 13

Q5. Online Search for Dynamic Warehouse Robot

1. Problem Statement

A warehouse robot must reach a destination while obstacles may appear unexpectedly. Since the environment changes, the robot must continuously observe the environment and modify its route.

2. Problem Analysis

- Initial State: Robot's current position
- Goal State: Target location
- Actions: Up, Down, Left, Right
- Problem: Unexpected obstacles
- Environment: Dynamic

3. AI Technique Selection

An Online Search Agent using LRTA* is selected. The robot observes the environment, chooses an action, moves, detects changes, updates its knowledge, and replans.

4. Heuristic

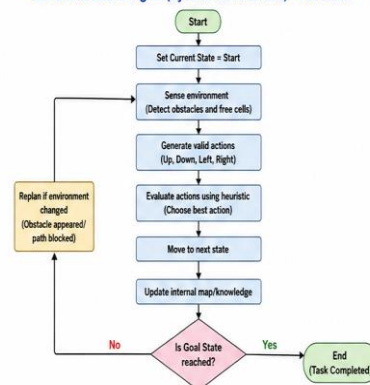
For a grid environment, use Manhattan distance: $h(n) = |x_n - x_g| + |y_n - y_g|$. Example: current (1,1), goal (3,4) $\rightarrow h(n) = |1-3| + |1-4| = 5$.

5. Algorithm / Pseudocode

- Set Current = Start.
- Sense the environment.
- Generate valid neighboring states.
- Calculate estimated cost.
- Select the best action and move to the selected state.

Flowchart: Online Search Agent (LRTA*)

Q5. Online Search Agent (Dynamic Environment) – Flowchart



Warehouse Grid

```

+---+---+---+---+
| S |   |   |   |
+---+---+---+---+
|   | X |   |   |
+---+---+---+---+
|   |   |   | G |
+---+---+---+---+

```

S=Start G=Goal X=Obstacle

- Detect new obstacles; update the map and heuristic.
- Replan if required.
- Repeat until Goal is reached.

6. Working

Initially the robot may choose $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$. Suppose an obstacle suddenly appears at B, blocking $S \rightarrow A \rightarrow X \rightarrow C \rightarrow G$.

The robot detects the obstacle, updates its map, and chooses an alternative: $S \rightarrow A \rightarrow D \rightarrow E \rightarrow C \rightarrow G$, adapting without continuing into the blocked location.

7. Complexity

If b is the number of possible actions and d is the number of steps — Time: approximately $O(bd)$ for local decisions; Space: $O(n)$, where n is the number of learned states.

8. Advantages and Limitations

- Advantage: Works in dynamic environments and adapts to new obstacles.
- Advantage: Does not require complete environmental knowledge; suitable for real-time robots.
- Limitation: May not always find the globally shortest path.
- Limitation: Replanning increases travel cost; requires sensors.

9. Conclusion

An Online Search Agent is more suitable than a static search algorithm for a warehouse with unexpected obstacles. It continuously observes the environment and changes its route when necessary: Dynamic Environment $\Rightarrow$ Online Search.

Comparison: A* vs Online Search

Feature	A*	Online
Static map	Great	Good
Dynamic env	Weak	Great
Replanning	Manual	Continuous
Full map	Needed	Not needed

Replanned Route

Original: S-A-B-C-G
 Blocked at B (now X)
 New: S-A-D-E-C-G

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding		15% 13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification		15% 13–15% – Selects the most appropriate AI algorithm /search	10–12% – Selects an appropriate algorithm	7–9% – Selects a partially suitable algorithm with limited	0–6% – Incorrect algorithm selection or no justification.

		technique with strong technical justification.	with adequate justification.	justification.	
Algorithm Design / Pseudocode		25% 22–25% – Designs a complete, logical, efficient, and error-free algorithm /pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax,		15% 13–15% – Correctly applies the relevant AI	10–12% – Applies AI concepts	7–9% – Demonstrates limited application of AI	0–6% – Unable to apply the appropriate AI

Alpha-Beta, etc.)		concepts and techniques throughout the solution.	correctly with minor mistakes.	concepts.	concepts or techniques.
--------------------------	--	---	---------------------------------------	------------------	--------------------------------

Diagram / State Space / Search Tree Representation	10% 9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10% 9–10% – Correctly analyzes time and space complexity and	7–8% – Complexity analysis is mostly correct	5–6% – Partial complexity analysis or limited	0–4% – No meaningful complexity analysis or highly inefficient solution.

	proposes an efficient solution.	with minor errors.	discussion of efficiency.	
Originality, Critical Thinking, and Presentation	10% 9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.